Machine learning on a cluster → library: SparkML

Machine learning library (MLlib) → Spark 3.5

org.apache.spark.ml

Mllib → allows to perform practical machine learning operations to be performed in a scalable computing environment(

https://spark.apache.org/docs/latest/ml-classification-regression.html

It offers tools such as:

ML algorithms: such as classification, regression, clustering and filtering

allowing to describe the **covariation** of several variables by fitting functions to them. It allows **predicting** unknown values of some quantities based on known values of others.

Functions: feature extraction, transformation, dimensionality reduction and selection

Pipelines: tools for constructing, evaluating and tuning ML Pipelines

Persistence: saving and load algorithms, models, and Pipelines

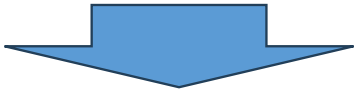
Utilities: linear algebra, statistics, data handling, etc.

The basic API (since Spark 3.2) is based on **DataFrame** (based on **SparkSQL**) for maintenance reasons, it is recommended to use **DF** instead of **RDD**

DataFrame-based API for MLlib provides a unified API for ML algorithms and multiple languages.

DataFrames make practical creation of ML pipelines, especially feature transformations, easier

Question:



Why is **MLlib** switching to the **DataFrame**-based API?

Question:

Why is MLlib switching to the DataFrame-based API?

The Answer is…

DataFrames provide a more user-friendly API than RDDs. The many benefits of **DataFrames** include **Spark Datasources**, **SQL/DataFrame queries**, Catalyst optimizations, and uniform APIs across languages.

DataFrames make practical creation of ML **pipelines**, especially feature transformations, easier

Is RDD-based MLlib **deprecated**?

MLlib includes both an **RDD-based API** and a **DataFrame-based API**. The **RDD**-based API is now in maintenance mode. But neither the API is deprecated, nor is **MLlib** as a whole.

Stages of Building a Classification Model

2. Exploratory analysis:

An example of how these statistics can be interpreted:

Let's assume we analyze feature1 from the dataset:

Statystyka	Wartość
Średnia	50.5
Odchylenie standardowe	10.2
Minimalna	12
Maksymalna	85

Interpretation:

Mean (50.5):→	Most attribute values hover around 50.
Standard deviation (10.2): →	Most values are within ±10 from the mean (i.e. 40–60).
Min value (12) and Max Value(85): →	The data covers a wide range and so may require normalization.

Stages of Building a Classification Model

3. Development of the model

A. Choosing the right classification algorithm →

- Random Forest
- Logistic Regression
- other…

4. Model evaluation (testing the model and assessment of its quality)

A. Splitting data into a SUB-set: →

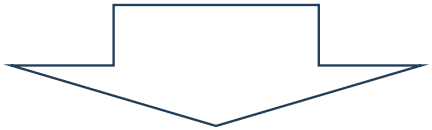
- subset for training and
- subset for testing

B. Determination of assessment measures: →

- Accuracy (precision)
- F1-score
- Confusion matrix

4. Model implementation and monitoring.

Stages of Building a Classification Model



Example: Letter Recognition

http://archive.ics.uci.edu/ml/datasets/Letter+Recognition

Question:



What is required at the very beginning to build an ML model?

Question:



What is required at the very beginning to build an ML model?

The answer is…



In order to carry out model construction

it is required to know all the features (attributes) of the set as well as the data itself is required

http://archive.ics.uci.edu/ml/datasets/Letter+Recognition

UCI Machine Learning Repository
Center for Machine Learning and Intelligent Systems

Letter Recognition Data Set
Download: Data Folder, Data Set Description

Data Set Information:

The objective is to identify each of a large number of black-and-white rectangular pixel displays as one of the 26 capital letters in the English alphabet. The character images were based on 20 different fonts and each letter within these 20 fonts was randomly distorted **to produce a file of 20,000 unique stimuli**. Each stimulus was converted into 16 primitive numerical attributes (statistical moments and edge counts) which were then scaled to fit into a range of integer values from 0 through 15.

We train the model on the first 16,000 elements, and then we use the resulting model to predict letter categories for the remaining 4,000.

multidimensional set

number of letter occurrences

Data Set Characteristics:	Multivariate	**Number of Instances:**	20000	**Area:**	Computer
Attribute Characteristics:	Integer	**Number of Attributes:**	16	**Date Donated**	1991-01-01
Associated Tasks:	Classification	**Missing Values?**	No	**Number of Web Hits:**	444476

for classification tasks

Attributes represented by int numbers.

Are there any missing values requiring:

filtering or

replenishment

number of attributes in the set of letters

Attribute Information:

1. lettr capital letter (26 values from A to Z)
2. x-box horizontal position of box (integer)
3. y-box vertical position of box (integer)
4. width width of box (integer)
5. high height of box (integer)
6. onpix total # on pixels (integer)
7. x-bar mean x of on pixels in box (integer)
8. y-bar mean y of on pixels in box (integer)
9. x2bar mean x variance (integer)
10. y2bar mean y variance (integer)
11. xybar mean x y correlation (integer)
12. x2ybr mean of x * x * y (integer)
13. xy2br mean of x * y * y (integer)
14. x-ege mean edge count left to right (integer)
15. xegvy correlation of x-ege with y (integer)
16. y-ege mean edge count bottom to top (integer)
17. yegvx correlation of y-ege with x (integer)

What does the data in this set look like?

Machine learning library (MLlib) → Spark 4.x

… is not as extensive as other well-known libraries available on the market, e.g.:

- PyTorch
- TensorFlow
- Scikit-learn
- Theano
- Keras

Its advantage → is the ability to work on a cluster with Spark installed with large data volumes.

DataFrame-based API for **MLlib** provides a unified API for **ML algorithms** and multiple languages.

```
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("ml").getOrCreate()
df = spark.read.csv("letter-recognition.data", header=False, inferSchema=True)
df.show(10)
df.schema
```

It starts in a similar way with a spark session.
Spark Context – This is the entry point to DF processing

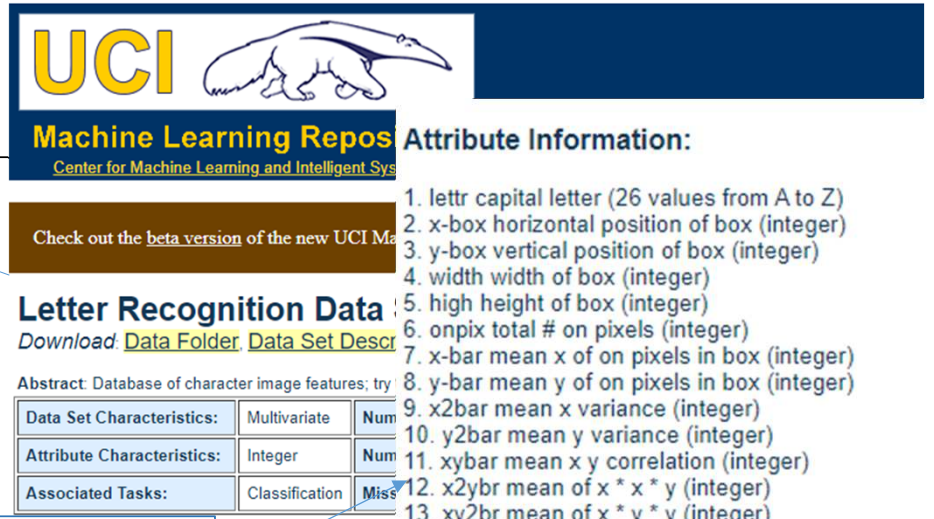
http://archive.ics.uci.edu/ml/datasets/Letter+Recognition

```
+---+---+---+---+---+---+---+---+---+---+----+----+----+----+----+----+----+
|_c0|_c1|_c2|_c3|_c4|_c5|_c6|_c7|_c8|_c9|_c10|_c11|_c12|_c13|_c14|_c15|_c16|
+---+---+---+---+---+---+---+---+---+---+----+----+----+----+----+----+----+
|  T|  2|  8|  3|  5|  1|  8| 13|  0|  6|   6|  10|   8|   0|   8|   0|   8|
|  I|  5| 12|  3|  7|  2| 10|  5|  5|  4|  13|   3|   9|   2|   8|   4|  10|
|  D|  4| 11|  6|  8|  6| 10|  6|  2|  6|  10|   3|   7|   3|   7|   3|   9|
|  N|  7| 11|  6|  6|  3|  5|  9|  4|  6|   4|   4|  10|   6|  10|   2|   8|
|  G|  2|  1|  3|  1|  1|  8|  6|  6|  6|   6|   5|   9|   1|   7|   5|  10|
|  S|  4| 11|  5|  8|  3|  8|  8|  6|  9|   5|   6|   6|   0|   8|   9|   7|
|  B|  4|  2|  5|  4|  4|  8|  7|  6|  6|   7|   6|   6|   2|   8|   7|  10|
|  A|  1|  1|  3|  2|  1|  8|  2|  2|  2|   8|   2|   8|   1|   6|   2|   7|
|  J|  2|  2|  4|  4|  2| 10|  6|  2|  6|  12|   4|   8|   1|   6|   1|   7|
|  M| 11| 15| 13|  9|  7| 13|  2|  6|  2|  12|   1|   9|   8|   1|   1|   8|
+---+---+---+---+---+---+---+---+---+---+----+----+----+----+----+----+----+
```

How to run:

```
python.exe .\ml_1.py
```

The set contains metrics that describe the data where letters appear in terms of their features in the image



Machine learning library (MLlib) → Spark 4.x

```
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("ml").getOrCreate()
df = spark.read.csv("letter-recognition.data", header=False, inferSchema=True)
df.show(10)
print(df.schema)
```

There is no header

The scheme is expected to be proposed by Spark

Describes the schema for data stored in a CSV file

```
StructType(List(StructField(_c0,StringType,true),StructField(_c1,IntegerType,true),StructField(_c2,IntegerType,true),StructFiel
d(_c3,IntegerType,true),StructField(_c4,IntegerType,true),StructField(_c5,IntegerType,true),StructField(_c6,IntegerType,true),S
tructField(_c7,IntegerType,true),StructField(_c8,IntegerType,true),StructField(_c9,IntegerType,true),StructField(_c10,IntegerTy
pe,true),StructField(_c11,IntegerType,true),StructField(_c12,IntegerType,true),StructField(_c13,IntegerType,true),StructField(_
c14,IntegerType,true),StructField(_c15,IntegerType,true),StructField(_c16,IntegerType,true)))
```

This allows you to define your own schema. Which after importing will be able to be precisely matched

letters

```
+---+---+---+---+---+---+---+---+---+---+----+----+----+----+----+----+----+
|_c0|_c1|_c2|_c3|_c4|_c5|_c6|_c7|_c8|_c9|_c10|_c11|_c12|_c13|_c14|_c15|_c16|
+---+---+---+---+---+---+---+---+---+---+----+----+----+----+----+----+----+
|  T|  2|  8|  3|  5|  1|  8| 13|  0|  6|   6|  10|   8|   0|   8|   0|   8|
|  I|  5| 12|  3|  7|  2| 10|  5|  5|  4|  13|   3|   9|   2|   8|   4|  10|
|  D|  4| 11|  6|  8|  6| 10|  6|  2|  6|  10|   3|   7|   3|   7|   3|   9|
|  N|  7| 11|  6|  6|  3|  5|  9|  4|  6|   4|   4|  10|   6|  10|   2|   8|
|  G|  2|  1|  3|  1|  1|  8|  6|  6|  6|   6|   5|   9|   1|   7|   5|  10|
|  S|  4| 11|  5|  8|  3|  8|  8|  6|  9|   5|   6|   6|   0|   8|   9|   7|
|  B|  4|  2|  5|  4|  4|  8|  7|  6|  6|   7|   6|   6|   2|   8|   7|  10|
|  A|  1|  1|  3|  2|  1|  8|  2|  2|  2|   8|   2|   8|   1|   6|   2|   7|
|  J|  2|  2|  4|  4|  2| 10|  6|  2|  6|  12|   4|   8|   1|   6|   1|   7|
|  M| 11| 15| 13|  9|  7| 13|  2|  6|  2|  12|   1|   9|   8|   1|   1|   8|
+---+---+---+---+---+---+---+---+---+---+----+----+----+----+----+----+----+
```

Features of the letters that we will use for recognition

```
python.exe .\ml_1.py
```

View basic feature statistics

Machine learning library (MLlib) → Spark 4.x

There is no header

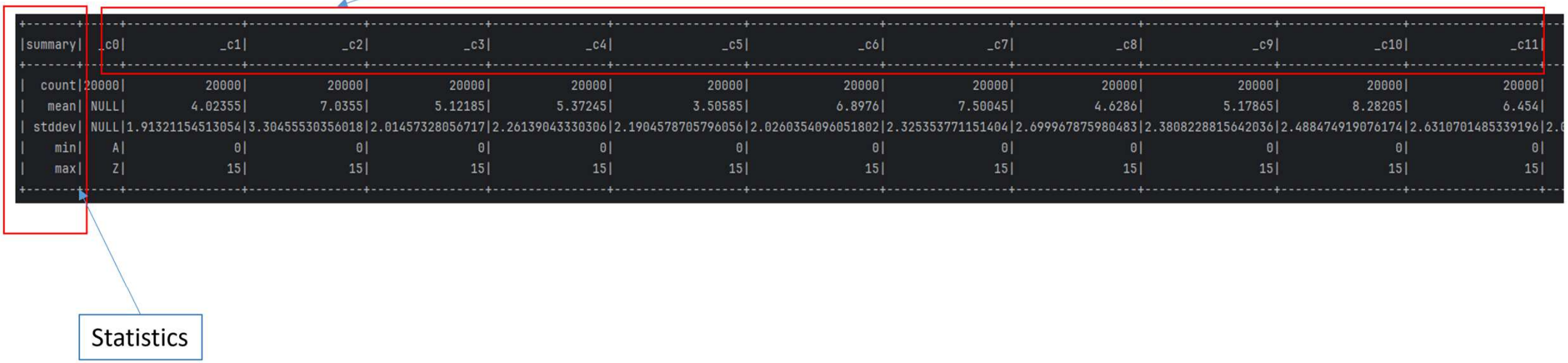
The scheme is expected to be proposed by Spark

```
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("ml").getOrCreate()
df = spark.read.csv("letter-recognition.data", header=False, inferSchema=True)
df.show(10)
print(df.schema)
# Allows you to display basic feature statistics
df.describe().show()
```

Allows you to display basic feature statistics

Features of the letters that we will use for recognition



summary	_c0	_c1	_c2	_c3	_c4	_c5	_c6	_c7	_c8	_c9	_c10	_c11
count	20000	20000	20000	20000	20000	20000	20000	20000	20000	20000	20000	20000
mean	NULL	4.02355	7.0355	5.12185	5.37245	3.50585	6.8976	7.50045	4.6286	5.17865	8.28205	6.454
stddev	NULL	1.91321154513054	3.30455530356018	2.01457328056717	2.26139043330306	2.1904578705796056	2.0260354096051802	2.325353771151404	2.699967875980483	2.3808228815642036	2.488474919076174	2.6310701485339196
min	A	0	0	0	0	0	0	0	0	0	0	0
max	Z	15	15	15	15	15	15	15	15	15	15	15

Statistics

python.exe .\ml_1.py

How to define a specific structure for a data schema?

The answer is..

The df.schema command provides a hint about the schema

See the previous slide

Machine learning library (MLlib) → Spark 4.x

```
from pyspark.sql import SparkSession
from pyspark.sql.types import StructType, StructField, StringType, IntegerType

schema = StructType([
    StructField("lettr", StringType(),  True),
    StructField("x_box", IntegerType(), True),
    StructField("y_box", IntegerType(), True),
    StructField("width", IntegerType(), True),
    StructField("high",  IntegerType(), True),
    StructField("onpix", IntegerType(), True),
    StructField("x_bar", IntegerType(), True),
    StructField("y_bar", IntegerType(), True),
    StructField("x2bar", IntegerType(), True),
    StructField("y2bar", IntegerType(), True),
    StructField("xybar", IntegerType(), True),
    StructField("x2ybr", IntegerType(), True),
    StructField("xy2br", IntegerType(), True),
    StructField("x_ege", IntegerType(), True),
    StructField("xegvy", IntegerType(), True),
    StructField("y_ege", IntegerType(), True),
    StructField("yegvx", IntegerType(), True),])
spark = SparkSession.builder.appName("ml").getOrCreate()
df = spark.read.csv("./letter/letter-recognition.data", header=False, schema=schema)
df.show(10)
df.schema
```

Imports to create your own schema

Allows you to provide **column names**

Providing the correct data types

The last attribute indicates whether missing data is allowed:

- True – may be
- FALSE – they can't be

Creating your own precisely defined structure of the schema and giving the columns names

Setting the schema for the data

Column names

Column types

Indicates NULL allowance in a column

ATTENTION → how to start!!!

From python 3.12 → you need to install the package:

```
pip install setuptools
pip install mumpy
```

Running the script:

```
python.exe .\ml_2.py
```

Running the script in python 3.11:

```
C:\"Program Files"\Python310\python.exe
C:\BigData\PycharmProjects\BigData\ml_71.py
```

ATTENTION !!! → **how to start :**

It is described in the package documentation

https://peps.python.org/pep-0632/

ModuleNotFoundError: No module named 'distutils'

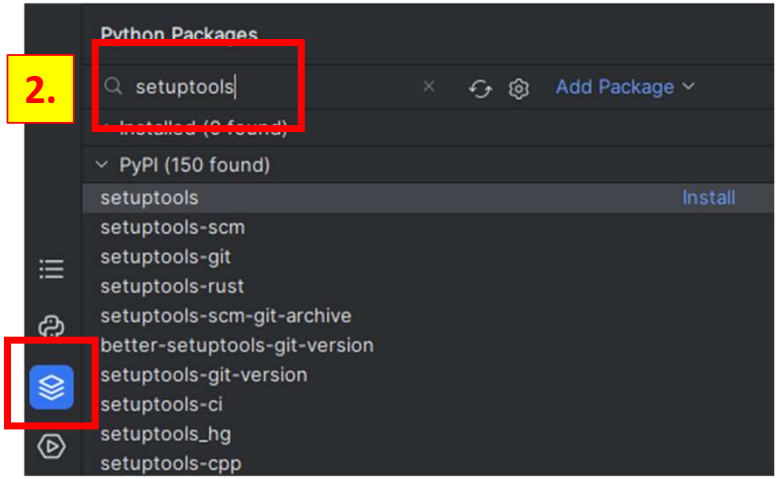
PEP 632 – distutils module is deprecated

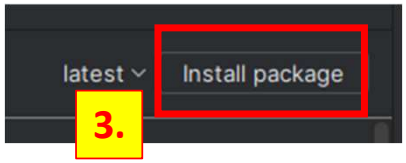
From python 3.12 → you need to install the package:

You don't need to reinstall pyspark and python → you only need to install **setuptools**

or… → in the environment: pyCharm:

pip install setuptools





Starting the task:

python.exe .\ml_2.py

Running the script in python 3.11:

C:\"Program Files"Python310\python.exe C:\BigData\PycharmProjects\BigData\ml_2.py

Note the space → the directory name is given in the „ " sign

Machine learning library (MLlib) → Spark 4.x

```
from pyspark.sql import SparkSession
from pyspark.sql.types import StructType, StructField, StringType, IntegerType
spark = SparkSession.builder.appName("ml").getOrCreate()
schema = StructType([
   StructField("lettr", StringType(),  True),
   StructField("x_box", IntegerType(), True),
   StructField("y_box", IntegerType(), True),
   StructField("width", IntegerType(), True),
   StructField("high",  IntegerType(), True),
   StructField("onpix", IntegerType(), True),
   StructField("x_bar", IntegerType(), True),
   StructField("y_bar", IntegerType(), True),
   StructField("x2bar", IntegerType(), True),
   StructField("y2bar", IntegerType(), True),
   StructField("xybar", IntegerType(), True),
   StructField("x2ybr", IntegerType(), True),
   StructField("xy2br", IntegerType(), True),
   StructField("x_ege", IntegerType(), True),
   StructField("xegvy", IntegerType(), True),
   StructField("y_ege", IntegerType(), True),
   StructField("yegvx", IntegerType(), True),
])
df = spark.read.csv("./letter/letter-recognition.data", header=False, schema=schema)
df.show(10)
a = df.count()
print(f"łączna liczba krotek wynosi: {a}")
df.createOrReplaceTempView("df")
spark.sql("SELECT lettr, COUNT(*) count FROM df GROUP BY lettr ORDER BY lettr").show(10)
```

The same code rearranged:

The names of the columns for processing have been created

```
+-----+-----+-----+-----+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|lettr|x_box|y_box|width|high|onpix|x_bar|y_bar|x2bar|y2bar|xybar|x2ybr|xy2br|x_ege|xegvy|y_ege|yegvx|
+-----+-----+-----+-----+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|    T|    2|    8|    3|   5|    1|    8|   13|    0|    6|    6|   10|    8|    0|    8|    0|    8|
|    I|    5|   12|    3|   7|    2|   10|    5|    5|    4|   13|    3|    9|    2|    8|    4|   10|
|    D|    4|   11|    6|   8|    6|   10|    6|    2|    6|   10|    3|    7|    3|    7|    3|    9|
|    N|    7|   11|    6|   6|    3|    5|    9|    4|    6|    4|    4|   10|    6|   10|    2|    8|
|    G|    2|    1|    3|   1|    1|    8|    6|    6|    6|    6|    5|    9|    1|    7|    5|   10|
|    S|    4|   11|    5|   8|    3|    8|    8|    6|    9|    5|    6|    6|    0|    8|    9|    7|
|    B|    4|    2|    5|   4|    4|    8|    7|    6|    6|    7|    6|    6|    2|    8|    7|   10|
|    A|    1|    1|    3|   2|    1|    8|    2|    2|    2|    8|    2|    8|    1|    6|    2|    7|
|    J|    2|    2|    4|   4|    2|   10|    6|    2|    6|   12|    4|    8|    1|    6|    1|    7|
|    M|   11|   15|   13|   9|    7|   13|    2|    6|    2|   12|    1|    9|    8|    1|    1|    8|
+-----+-----+-----+-----+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
```

20 000 tuples

VIDE slide next

It will allow us to register our **df** as a temporal view so that we can still perform transformations on the data using **Spark SQL**

2. Label Distribution Analysis

To see how often particular labels (letters) appear, you can use: the functions: **groupBy()** and **count().**

count() - is an **action** (unlike a **transformation**), so it returns a non-DataFrame object
- in this case **int type** representing the number of rows in **DataFrame**.
- **Int** it doesn't have a method called show() in it → for this reason we pass the value to the variable and display it further

```
...
df = spark.read.csv("./letter/letter-recognition.data", header=False, schema=schema)
a = df.count()
print(f"łączna liczba krotek wynosi: {a}")
df.createOrReplaceTempView("df")
spark.sql("SELECT lettr, COUNT(*) count FROM df GROUP BY lettr ORDER BY lettr").show(10)
```

It will allow us to register our **df** as a temporal view so that we can still perform transformations on the data using **Spark SQL**

```
łączna liczba krotek wynosi: 20000
+-----+-----+
|lettr|count|
+-----+-----+
|    A|  789|
|    B|  766|
|    C|  736|
|    D|  805|
|    E|  768|
|    F|  775|
|    G|  773|
|    H|  734|
|    I|  755|
|    J|  747|
+-----+-----+
only showing top 10 rows
```

Aggregation values against letter data

3. Analysis of feature distribution

You can explore the distribution of each numerical feature (e.g. feature1, feature2) using the functions: **select()** and **rdd.flatMap()**

Own work

4. Correlation between features

To understand the correlations between features, you can use a correlation matrix.

Own work

4. Correlation between features

To understand the correlations between features, you can use a correlation matrix.

Apache Spark and MLlib library → allows you to calculate correlations between columns of data.

```
from pyspark.ml.feature import VectorAssembler
from pyspark.ml.stat import Correlation
from pyspark.sql.types import StructType, StructField, IntegerType, StringType
```

```
assembler = VectorAssembler(
    inputCols=[
        "x_box", "y_box", "width", "high", "onpix",
        "x_bar", "y_bar", "x2bar", "y2bar", "xybar",
        "x2ybr", "xy2br", "x_ege", "xegvy", "y_ege", "yegvx"
    ],
    outputCol="features"
)
vector_data = assembler.transform(df)
```

Calculation of the correlation matrix:

```
correlation_matrix = Correlation.corr(vector_data, "features").head()[0]
```

Converting a matrix to a NumPy array:

```
import numpy as np
corr_matrix_np = np.array(correlation_matrix.toArray())
```

4. Correlation between features

Displaying the correlation matrix:

```
print("correlation matrix:")
print(corr_matrix_np)
```

```
Macierz korelacji:
[[ 1.          0.7577928   0.851514    0.67276367  0.61909688 -0.03259473
   0.04554493  0.01430618  0.05208644  0.14805631  0.03546375 -0.04633326
   0.48915474  0.09817988  0.27350437 -0.10514719]
 [ 0.7577928   1.          0.67191188  0.82320706  0.55506655  0.04568989
  -0.04092544 -0.02501924  0.09647792  0.15995434 -0.05464832 -0.00756773
   0.27443129 -0.00133618  0.23088331 -0.04274148]
 [ 0.851514    0.67191188  1.          0.66021536  0.76571612  0.06195861
   0.0248316  -0.09861074  0.05707374  0.11501821  0.01169388 -0.04500904
   0.55725093  0.04565754  0.26028484 -0.11827346]
 [ 0.67276367  0.82320706  0.66021536  1.          0.64436627  0.04284448
  -0.02007174  0.08238279  0.05903222  0.01245759 -0.01199115  0.02638634
   0.26524301  0.02535914  0.29754496 -0.01885332]
 [ 0.61909688  0.55506655  0.76571612  0.64436627  1.          0.13915898
  -0.0288225  -0.01198493 -0.06555727 -0.06977638 -0.07294097 -0.03885753
   0.62750675  0.01764911  0.49265252 -0.06296872]
 [-0.03259473  0.04568989  0.06195861  0.04284448  0.13915898  1.
  -0.35658015 -0.05330597 -0.12285057  0.08596313 -0.34195727 -0.03211485
   0.14432546 -0.2533388   0.12705649  0.24881624]
 [ 0.04554493 -0.04092544  0.0248316  -0.02007174 -0.0288225  -0.35658015
   1.         -0.11862456 -0.04965796  0.17831757  0.60039686 -0.27164865
  -0.03672222  0.55506025 -0.0780084  -0.20789953]
 [ 0.01430618 -0.02501924 -0.09861074  0.08238279 -0.01198493 -0.05330597
  -0.11862456  1.         -0.18843071 -0.31778035  0.04254491  0.08201959
   0.14213192 -0.08482047  0.00654568  0.18290226]
 [ 0.05208644  0.09647792  0.05707374  0.05903222 -0.06555727 -0.12285057
  -0.04965796 -0.18843071  1.          0.13200022 -0.06011641  0.11904784
  -0.38401794 -0.05254493  0.27753999 -0.06133517]
 [ 0.14805631  0.15995434  0.11501821  0.01245759 -0.06977638  0.08596313
   0.17831757 -0.31778035  0.13200022  1.          0.05798807 -0.10675902
  -0.17567557  0.02941897 -0.08701903 -0.11422301]
 [ 0.03546375 -0.05464832  0.01169388 -0.01199115 -0.07294097 -0.34195727
   0.60039686  0.04254491 -0.06011641  0.05798807  1.          0.06321432
   0.05356572  0.52723942 -0.22625073 -0.23651821]
 [-0.04633326 -0.00756773 -0.04500904  0.02638634 -0.03885753 -0.03211485
  -0.27164865  0.08201959  0.11904784 -0.10675902  0.06321432  1.
  -0.00875291 -0.18492713  0.04969529  0.2458078 ]
 [ 0.48915474  0.27443129  0.55725093  0.26524301  0.62750675  0.14432546
  -0.03672222  0.14213192 -0.38401794 -0.17567557  0.05356572 -0.00875291
   1.          0.00284926  0.10841087 -0.04978901]
 [ 0.09817988 -0.00133618  0.04565754  0.02535914  0.01764911 -0.2533388
   0.55506025 -0.08482047 -0.05254493  0.02941897  0.52723942 -0.18492713
   0.00284926  1.         -0.06440225 -0.18759068]
 [ 0.27350437  0.23088331  0.26028484  0.29754496  0.49265252  0.12705649
  -0.0780084   0.00654568  0.27753999 -0.08701903 -0.22625073  0.04969529
   0.10841087 -0.06440225  1.          0.14358774]
 [-0.10514719 -0.04274148 -0.11827346 -0.01885332 -0.06296872  0.24881624
  -0.20789953  0.18290226 -0.06133517 -0.11422301 -0.23651821  0.2458078
  -0.04978901 -0.18759068  0.14358774  1.        ]]
```

4. Correlation between features

Own work

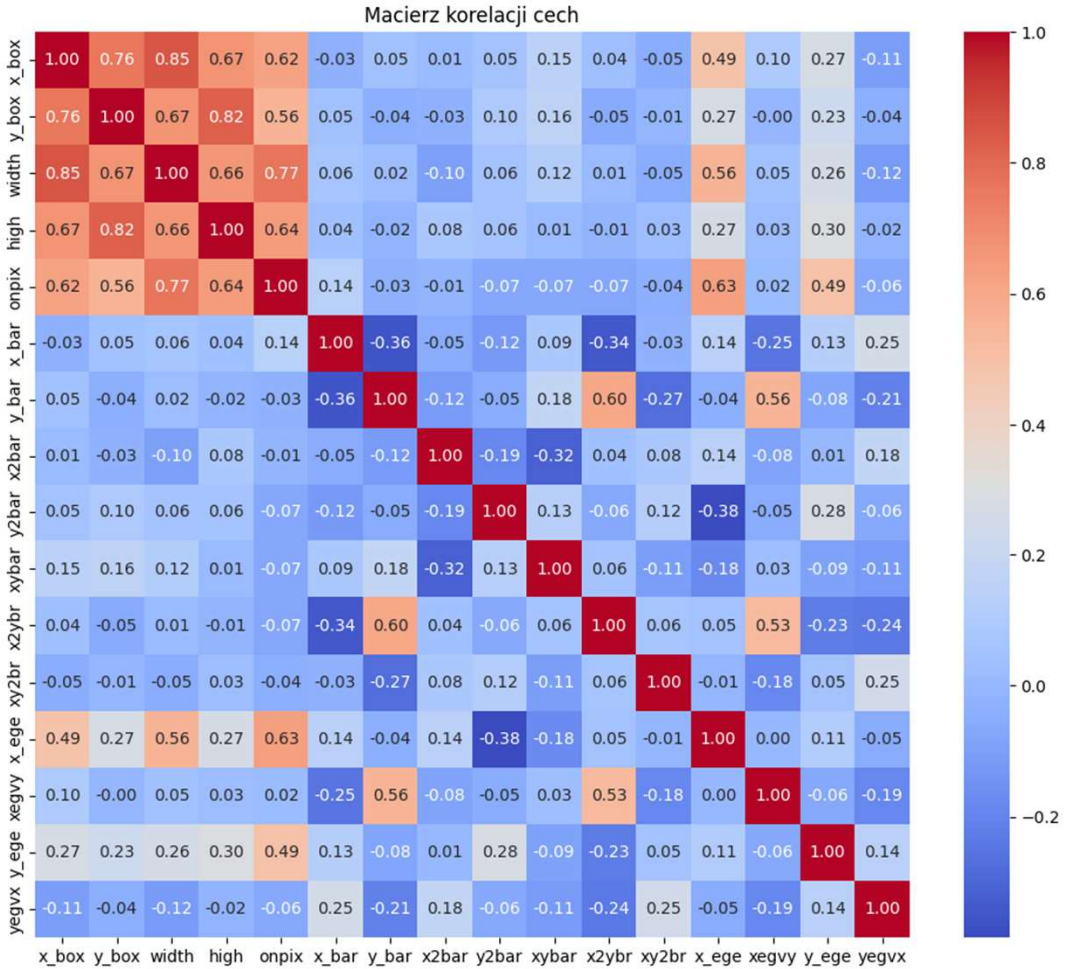
Correlation Matrix Visualization:

```
import seaborn as sns
import matplotlib.pyplot as plt

# Correlation Matrix Visualization:
plt.figure(figsize=(12, 10))
sns.heatmap(corr_matrix_np, annot=True, fmt=".2f", cmap="coolwarm",
            xticklabels=[
                "x_box", "y_box", "width", "high", "onpix",
                "x_bar", "y_bar", "x2bar", "y2bar", "xybar",
                "x2ybr", "xy2br", "x_ege", "xegvy", "y_ege", "yegvx"
            ],
            yticklabels=[
                "x_box", "y_box", "width", "high", "onpix",
                "x_bar", "y_bar", "x2bar", "y2bar", "xybar",
                "x2ybr", "xy2br", "x_ege", "xegvy", "y_ege", "yegvx"
            ])
plt.title("Correlation Matrix Visualization")
plt.show()
```

4. Correlation between features

Correlation Matrix Visualization:



explain the obtained graph of the feature correlation matrix. What might it be needed for?

Own work

5. Conclusions from the preliminary data analysis

Label (letter) layout: → **Q:** Is there a class imbalance?

This should be taken into account in later steps, e.g. by weighting the classes in the model.

Distribution of features: → **Q:** Does the data contain outliers?

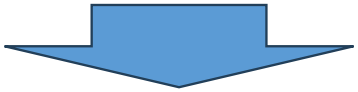
It is worth analyzing them and potentially removing them.

Correlations: → **Q:** Are there any highly correlated features?

This can be used to reduce the dimensionality of the data.

Own work

Question:



Why is **MLlib** switching to the **DataFrame**-based API?

Question:

Why is MLlib switching to the DataFrame-based API?

The Answer is…

DataFrames provide a more user-friendly API than RDDs. The many benefits of **DataFrames** include **Spark Datasources**, **SQL/DataFrame queries**, Catalyst optimizations, and uniform APIs across languages.

DataFrames make practical creation of ML **pipelines**, especially feature transformations, easier

Is RDD-based MLlib **deprecated**?

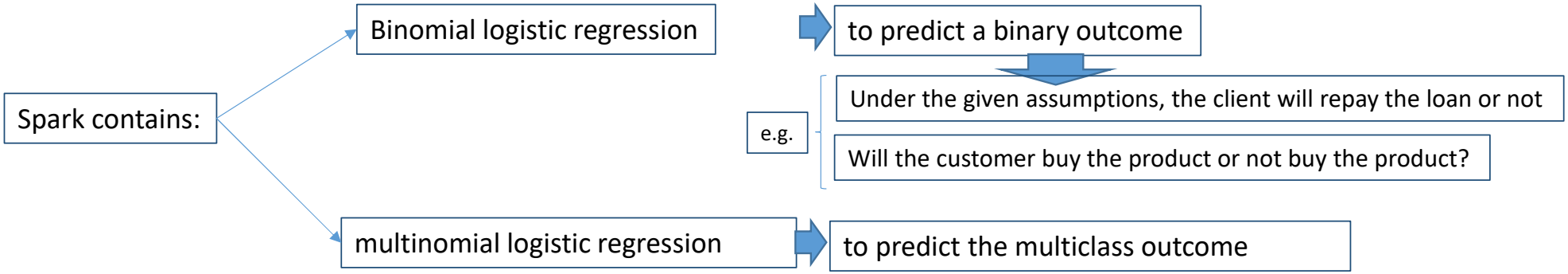
MLlib includes both an **RDD-based API** and a **DataFrame-based API**. The **RDD**-based API is now in maintenance mode. But neither the API is deprecated, nor is **MLlib** as a whole.

Classification and Logistic Regression

https://spark.apache.org/docs/latest/ml-classification-regression.html

Logistic regression

is a popular method to predict a categorical response. It is a special case of Generalized Linear models that predicts the probability of the outcomes. This is a special case of generalized linear models that predict the probability of outcomes (e.g. based on the description of the features of this object)



Regression → The method involves dividing the set and labeled data that will be used to build a model (**we need to have results in such data**, i.e. we need to know, for example, whether the customer repaid the loan or not based on the given characteristics)

This allows you to train the model on data and results that have already occurred

In order to answer the question whether a new person can be classified in the group of customers with good prospects for repaying such a loan.

Construction of the model

https://spark.apache.org/docs/latest/ml-classification-regression.html

Model selection (also known as hyperparameter tuning)

An important task in ML is model selection, that is, using data to find the best model or parameters for a given task.

This is also called → *tuning*

The structure of the model allows you to learn the relationships between the features assigned to classes of objects

tuning can be performed for individual estimators (**Estimators**), such as LogisticRegression, or for entire so-called **Pipelines** that contain multiple algorithms, functions, and other tuples. This allows the model builder to tune the entire Pipeline, rather than tuning each Pipeline element individually.(

Estimators → **fit models to data.** → an algorithm that creates a **transformer** (**fit**) based on **DF**

Evaluators → **they calculate metrics based on predictions for the scoring process**

CrossValidator extends Estimator → K-fold cross-validation performs model selection by dividing the dataset into a set of randomly split data collections that do not overlap and that are used as separate training and test data sets, e.g. with k=3, k-fold cross-validation will generate 3 pairs datasets (training, test), each of which uses 2/3 of the data for training and 1/3 for testing. Each assembly is used as a test set exactly once.

MLlib supports model selection using objects:

- CrossValidator
- TrainValidationSplit

ML Pipelines → provides a uniform set of high-level APIs built on top of DataFrames that help users create and customize actionable machine learning pipelines.

Model training

https://spark.apache.org/docs/latest/ml-classification-regression.html

Estimator → fit the model to the data. → an algorithm that creates a **transformer** (fit) based on **DF**

Causes initial initialization of an untrained data model → training in Spark involves calling the fit method to which we will pass a **DataFrame** (i.e. data)

As a result of calling the **fit** method after training the model, we will receive **a new model**, which is the **Transformer**.

Transformer → an algorithm that transforms the input DF into another DF, e.g. a trained ML model creating a new DF containing predictions (transforms)

In order for Transformer to be able to generate a prediction, we need the transform method

Transformer → serial connection of transformers and estimators to create a workflow

Model prediction

Forecasting or prediction → a method of predicting how processes or events will develop in the future. During the forecasting process, a judgment is made about the future states of phenomena and events, called a forecast.

… Forecasting examines the relationships between these factors and the phenomenon under study and how they (these phenomena) developed in the past to draw conclusions about the future.

Using pySpark, consider how to develop further steps to build a classification model



Take into account the division of the set into a set intended for training (learning) and testing.

ml_5.py

pip install numpy==numpy-1.21.4

The process begins with dividing the data set into two parts that will be used for training and prediction

```
df = spark.read.csv("./letter/letter-recognition.data" header=False, schema=schema)
df.createOrReplaceTempView("df")
df_train, df_eval = df.randomSplit([0.7, 0.3], 42)
from pyspark.ml import feature
idx = feature.StringIndexer(inputCol="lettr", outputCol="label")
idx_t = idx.fit(df_train)
df_train_ = idx_t.transform(df_train)
df_train_.show(10)
```

It divides the data into two parts and allows you to divide df in the proportions of 70% by 30%. Value 42 – this is the randomization seed (to obtain a deterministic division).

Part → intended for training

Part → intended for checking the results on new, unused data

A module that provides tools for data preparation and scaling, normalization support for further model training.

https://spark.apache.org/docs/latest/api/python/reference/pyspark.ml.html#feature

The method that is responsible for creating a column represented by numbers from a column containing String. Allows you to specify an input column and an output column. The output will map the string to a unique numeric value.

Question: **...what is the purpose of reducing the values of the String type to unique numeric values of the Int type?**

The handle is the estimator object (its object is created)

answer: **...all model building operations performed → are matrix operations, and matrix transformation operations are best performed on numerical values**

The **fit** method to which we pass the **df** object will allow us to create a transformer object, i.e. to transform our data.

lettr	x_box	y_box	width	high	onpix	x_bar	y_bar	x2bar	y2bar	xybar	x2ybr	xy2br	x_ege	xegvy	y_ege	yegvx	label
A	1	0	2	0	0	7	3	2	0	7	2	8	2	6	1	8	0.0
A	1	0	2	0	0	7	3	2	0	7	2	8	2	6	1	8	0.0
A	1	0	2	0	0	7	4	2	0	7	2	8	1	6	1	8	0.0
A	1	0	2	0	0	7	4	2	0	7	2	8	1	7	1	8	0.0

A new column is created with a unique (for each letter) numerical marking for given object classes, e.g. A→ 0.0

VIDE slide next

ml_6.py

The purpose of using numerical values is to enable matrix operations

For this reason, all input features and classes (here letters) should be numeric values

lettr	x_box	y_box	width	high	onpix	x_bar	y_bar	x2bar	y2bar	xybar	x2ybr	xy2br	x_ege	xegvy	y_ege	yegvx	label
A	1	0	2	0	0	7	3	2	0	7	2	8	2	6	1	8	0.0
A	1	0	2	0	0	7	3	2	0	7	2	8	2	6	1	8	0.0
A	1	0	2	0	0	7	4	2	0	7	2	8	1	6	1	8	0.0
A	1	0	2	0	0	7	4	2	0	7	2	8	1	7	1	8	0.0

... in the next step, the input feature variables should be transformed into vectors

```
vect = feature.VectorAssembler(inputCols=df.columns[1:], outputCol="feat")
df_train_ = vect.transform(df_train_)
```

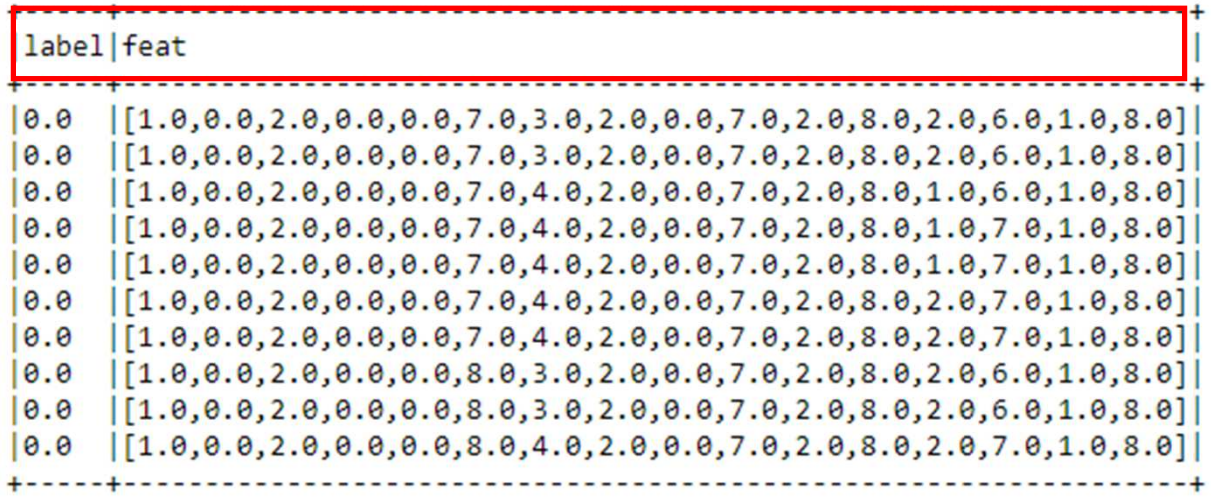
For this purpose, VectorAssembler will be used, which is a transformer that allows you to combine features and columns. It doesn't have to learn anything from the data. The output column will be called **feat**.

VIDE slide next

ml_6.py

Then we select only two columns from df

```
df_train_ = df_train_.select("label", "feat")
```



label	feat
0.0	[1.0,0.0,2.0,0.0,0.0,7.0,3.0,2.0,0.0,7.0,2.0,8.0,2.0,6.0,1.0,8.0]
0.0	[1.0,0.0,2.0,0.0,0.0,7.0,3.0,2.0,0.0,7.0,2.0,8.0,2.0,6.0,1.0,8.0]
0.0	[1.0,0.0,2.0,0.0,0.0,7.0,4.0,2.0,0.0,7.0,2.0,8.0,1.0,6.0,1.0,8.0]
0.0	[1.0,0.0,2.0,0.0,0.0,7.0,4.0,2.0,0.0,7.0,2.0,8.0,1.0,7.0,1.0,8.0]
0.0	[1.0,0.0,2.0,0.0,0.0,7.0,4.0,2.0,0.0,7.0,2.0,8.0,1.0,7.0,1.0,8.0]
0.0	[1.0,0.0,2.0,0.0,0.0,7.0,4.0,2.0,0.0,7.0,2.0,8.0,2.0,7.0,1.0,8.0]
0.0	[1.0,0.0,2.0,0.0,0.0,7.0,4.0,2.0,0.0,7.0,2.0,8.0,2.0,7.0,1.0,8.0]
0.0	[1.0,0.0,2.0,0.0,0.0,8.0,3.0,2.0,0.0,7.0,2.0,8.0,2.0,6.0,1.0,8.0]
0.0	[1.0,0.0,2.0,0.0,0.0,8.0,3.0,2.0,0.0,7.0,2.0,8.0,2.0,6.0,1.0,8.0]
0.0	[1.0,0.0,2.0,0.0,0.0,8.0,4.0,2.0,0.0,7.0,2.0,8.0,2.0,7.0,1.0,8.0]

The result of the statement is a transformation that will select only two columns from df

In the next step, this data should be normalized (i.e. scaled) so that the ranges of values match each other

ml_7.py

StandardScaler – normalizes the data (i.e. scales) so that the ranges of values match each other

```
scaler = feature.StandardScaler(inputCol="feat", outputCol="features")
scaler_t = scaler.fit(df_train_)
df_train_ = scaler_t.transform(df_train_)
```

StandardScaler – uses the mean and standard deviation of each of our columns.
This will allow you to normalize and scale the columns to have similar values. It is an estimator (because it has to learn the data. It takes an input and output column

```
+-----+--------------------+--------------------+
|label|                feat|            features|
+-----+--------------------+--------------------+
|  0.0|[1.0,0.0,2.0,0.0,...|[0.52088456476688...|
|  0.0|[1.0,0.0,2.0,0.0,...|[0.52088456476688...|
|  0.0|[1.0,0.0,2.0,0.0,...|[0.52088456476688...|
|  0.0|[1.0,0.0,2.0,0.0,...|[0.52088456476688...|
|  0.0|[1.0,0.0,2.0,0.0,...|[0.52088456476688...|
|  0.0|[1.0,0.0,2.0,0.0,...|[0.52088456476688...|
|  0.0|[1.0,0.0,2.0,0.0,...|[0.52088456476688...|
|  0.0|[1.0,0.0,2.0,0.0,...|[0.52088456476688...|
|  0.0|[1.0,0.0,2.0,0.0,...|[0.52088456476688...|
|  0.0|[1.0,0.0,2.0,0.0,...|[0.52088456476688...|
+-----+--------------------+--------------------+
```

Scaling results

... or: presentation Without text truncation:

```
df_train_.show(10, truncate=False)
```

How to implement classification in model building?

Classification:

is a process related to categorization…

… the process in which ideas and objects are recognized (differentiated).

Classification is the grouping of related facts into classes.

Model - classification:

LinearSVC(*[, featuresCol, labelCol, …])	This binary classifier optimizes the Hinge Loss using the OWLQN optimizer.
LinearSVCModel([java_model])	Model fitted by LinearSVC.
LinearSVCSummary([java_obj])	Abstraction for LinearSVC Results for a given model.
LinearSVCTrainingSummary([java_obj])	Abstraction for LinearSVC Training results.
LogisticRegression(*[, featuresCol, …])	Logistic regression.
LogisticRegressionModel([java_model])	Model fitted by LogisticRegression.
LogisticRegressionSummary([java_obj])	Abstraction for Logistic Regression Results for a given model.
LogisticRegressionTrainingSummary([java_obj])	Abstraction for multinomial Logistic Regression Training results.
BinaryLogisticRegressionSummary([java_obj])	Binary Logistic regression results for a given model.
BinaryLogisticRegressionTrainingSummary([…])	Binary Logistic regression training results for a given model.
DecisionTreeClassifier(*[, featuresCol, …])	Decision tree learning algorithm for classification.It supports both binary and multiclass labels, as well as both continuous and categorical features..
DecisionTreeClassificationModel([java_model])	Model fitted by DecisionTreeClassifier.
GBTClassifier(*[, featuresCol, labelCol, …])	Gradient-Boosted Trees (GBTs) learning algorithm for classification.It supports binary labels, as well as both continuous and categorical features..
GBTClassificationModel([java_model])	Model fitted by GBTClassifier.
RandomForestClassifier(*[, featuresCol, …])	Random Forest learning algorithm for classification.It supports both binary and multiclass labels, as well as both continuous and categorical features..
RandomForestClassificationModel([java_model])	Model fitted by RandomForestClassifier.
RandomForestClassificationSummary([java_obj])	Abstraction for RandomForestClassification Results for a given model.
RandomForestClassificationTrainingSummary([…])	Abstraction for RandomForestClassificationTraining Training results.
BinaryRandomForestClassificationSummary([…])	BinaryRandomForestClassification results for a

and other

https://spark.apache.org/docs/latest/api/python/reference/pyspark.ml.html#classification

Model - classification: RandomForestClassifier

Random Forest → learning algorithm for classification.

It supports both binary and multiclass labels, as well as both continuous and categorical features.

```
class pyspark.ml.classification.RandomForestClassifier(*, featuresCol: str = 'features',
labelCol: str = 'label', predictionCol: str = 'prediction', probabilityCol: str = 'probability',
rawPredictionCol: str = 'rawPrediction', maxDepth: int = 5, maxBins: int = 32,
minInstancesPerNode: int = 1, minInfoGain: float = 0.0, maxMemoryInMB: int = 256,
cacheNodeIds: bool = False, checkpointInterval: int = 10, impurity: str = 'gini', numTrees: int =
20, featureSubsetStrategy: str = 'auto', seed: Optional[int] = None, subsamplingRate: float =
1.0, leafCol: str = '', minWeightFractionPerNode: float = 0.0, weightCol: Optional[str] = None,
bootstrap: Optional[bool] = True)                                                    [source]
```

Selected attributes:

cacheNodeIds → the algorithm will pass trees to executors to match instances with nodes. If true, the algorithm will cache node IDs for each instance. Caching can speed up training of deeper trees. Users can set how often should the cache be checkpointed or disable it by setting checkpointInterval.')

maxBins → number of categories for any categorical feature.

maxDepth → E.g., depth 0 means 1 leaf node; depth 1 means 1 internal node + 2 leaf nodes. Must be in range [0, 30].

maxMemoryInMB →'Maximum memory in MB allocated to histogram aggregation. If too small, then 1 node will be split per iteration, and its aggregates may exceed this size.

numTrees → Number of trees to train (>= 1).

Methods

Vide:

https://spark.apache.org/docs/latest/api/python/reference/pyspark.ml.html#classification

ml_8.py

Model for classification: https://spark.apache.org/docs/latest/api/python/reference/pyspark.ml.html#classification

```
from pyspark.ml import classification
forest = classification.RandomForestClassifier(maxDepth=8, minInstancesPerNode=5, seed=42)
forest_t = forest.fit(df_train_)
pred_train = forest_t.transform(df_train_)
pred_train.show(10)
```

We will use a model **RandomForestClassifier** → is a type of model that consists of simpler decision tree models.

Methods (for the model) are selected or some techniques can be used that will improve learning. We provide the model depth and sampling seed.

Following this method, an estimator is created that has a **fit** method. The output will be a transformer in the form of a ready-made model.

Prediction is made using the transform operation.

pred_train

```
+-----+--------------------+--------------------+--------------------+--------------------+----------+
|label|                feat|            features|       rawPrediction|         probability|prediction|
+-----+--------------------+--------------------+--------------------+--------------------+----------+
|  0.0|[1.0,0.0,2.0,0.0,...|[0.52088456476688...|[19.7966643455730...|[0.98983321727865...|       0.0|
|  0.0|[1.0,0.0,2.0,0.0,...|[0.52088456476688...|[19.7966643455730...|[0.98983321727865...|       0.0|
|  0.0|[1.0,0.0,2.0,0.0,...|[0.52088456476688...|[18.8810217063951...|[0.94405108531975...|       0.0|
|  0.0|[1.0,0.0,2.0,0.0,...|[0.52088456476688...|[17.1194832448566...|[0.85597416224283...|       0.0|
|  0.0|[1.0,0.0,2.0,0.0,...|[0.52088456476688...|[17.1194832448566...|[0.85597416224283...|       0.0|
|  0.0|[1.0,0.0,2.0,0.0,...|[0.52088456476688...|[19.6595901989730...|[0.98297950994865...|       0.0|
+-----+--------------------+--------------------+--------------------+--------------------+----------+
```

Here's the prediction

How to perform model evaluation?

evaluation → allows you to assess the quality of the completed model

Model evaluation:

ml_9.py

Model evaluation:

The evaluation module → offers various evaluators

```
from pyspark.ml import evaluation
evaluator = evaluation.MulticlassClassificationEvaluator(metricName="accuracy")
a = evaluator.evaluate(pred_train)
print(f"Probability of correctly matching the letter: {a}")
df_eval_ = idx_t.transform(df_eval)
df_eval_ = vect.transform(df_eval_)
df_eval_ = df_eval_.select("label", "feat")
df_eval_ = scaler_t.transform(df_eval_)
pred_eval = forest_t.transform(df_eval_)
```

```
Prawdopodobieństwo prawidłowego dopasowania litery: 0.7926205033413906
```

NOTE → prediction on trained data

Prediction on new data (NEXT - you can replace using pipeline)

MulticlassClassificationEvaluator – allows the use of the accuracy metric, which tells us with what probability the predictions were consistent?

The evaluator offers the evaluate method

We provide data containing predictions to compare real data. The output will be data in the form of a prediction coefficient. If it is closer to 1, we have received a better model. The result is 79%

pred_eval.show(10)

```
+-----+--------------------+--------------------+--------------------+--------------------+----------+
|label|                feat|            features|       rawPrediction|         probability|prediction|
+-----+--------------------+--------------------+--------------------+--------------------+----------+
|  0.0|[1.0,0.0,2.0,0.0,...|[0.52088456476688...|[18.8810217063951...|[0.94405108531975...|       0.0|
|  0.0|[1.0,0.0,2.0,0.0,...|[0.52088456476688...|[19.6595901989730...|[0.98297950994865...|       0.0|
|  0.0|[1.0,0.0,2.0,0.0,...|[0.52088456476688...|[19.6595901989730...|[0.98297950994865...|       0.0|
|  0.0|[1.0,0.0,2.0,0.0,...|[0.52088456476688...|[19.6595901989730...|[0.98297950994865...|       0.0|
|  0.0|[1.0,0.0,2.0,0.0,...|[0.52088456476688...|[19.7730794399127...|[0.98865397199563...|       0.0|
|  0.0|[1.0,0.0,2.0,0.0,...|[0.52088456476688...|[18.8574368007347...|[0.94287184003673...|       0.0|
|  0.0|[1.0,0.0,2.0,0.0,...|[0.52088456476688...|[18.8574368007347...|[0.94287184003673...|       0.0|
|  0.0|[1.0,0.0,2.0,1.0,...|[0.52088456476688...|[19.6595901989730...|[0.98297950994865...|       0.0|
|  0.0|[1.0,0.0,2.0,1.0,...|[0.52088456476688...|[19.6595901989730...|[0.98297950994865...|       0.0|
|  0.0|[1.0,0.0,2.0,1.0,...|[0.52088456476688...|[19.6360052933126...|[0.98180026466563...|       0.0|
+-----+--------------------+--------------------+--------------------+--------------------+----------+
```

ml_9.py

Probability of matching a letter for the untrained set:

```
b= evaluator.evaluate(pred_eval)
print(f" Probability of correctly matching the letter: {b}")
```

```
Prawdopo. dopasowania litery dane dla zbioru nietrenowanego: 0.7595214020896528
```

therefore… → … A few percent less than for the training set

Model evaluation via pipeline:

ml_10.py

Model evaluation:

The pipline module → offers a universal data flow, so you can connect a data to stream

```
from pyspark.ml import Pipeline

idx = feature.StringIndexer(inputCol="lettr", outputCol="label")
vect = feature.VectorAssembler(inputCols=df.columns[1:], outputCol="feat")
scaler = feature.StandardScaler(inputCol="feat", outputCol="features")
forest = classification.RandomForestClassifier(maxDepth=8, minInstancesPerNode=5, seed=42)

pipe = Pipeline(stages=[idx, vect, scaler, forest])
pipe_t = pipe.fit(df_train)
pipe_t.transform(df_eval).drop(*df.columns[1:]).show(10)
```

Initializing the estimators and inserting them into the **pipe** stream, not initializing the steps but indicating that these will be states to be executed

The order of steps is very important and requires the order in which the steps will be performed

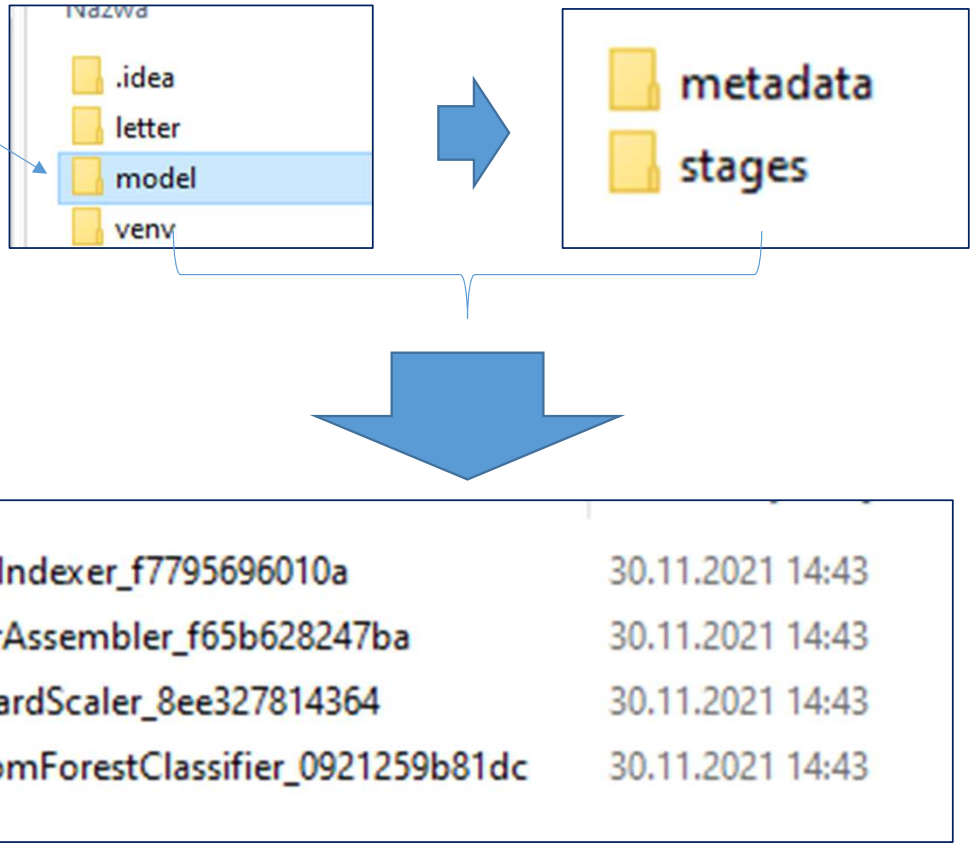
Pipe – is an estimator, so in the next step I use the fit method.

```
+-----+--------------------+--------------------+--------------------+--------------------+----------+
|label|                feat|            features|       rawPrediction|         probability|prediction|
+-----+--------------------+--------------------+--------------------+--------------------+----------+
|  0.0|[1.0,0.0,2.0,0.0,...|[0.52088456476688...|[18.8810217063951...|[0.94405108531975...|       0.0|
|  0.0|[1.0,0.0,2.0,0.0,...|[0.52088456476688...|[19.6595901989730...|[0.98297950994865...|       0.0|
|  0.0|[1.0,0.0,2.0,0.0,...|[0.52088456476688...|[19.6595901989730...|[0.98297950994865...|       0.0|
|  0.0|[1.0,0.0,2.0,0.0,...|[0.52088456476688...|[19.6595901989730...|[0.98297950994865...|       0.0|
|  0.0|[1.0,0.0,2.0,0.0,...|[0.52088456476688...|[19.7730794399127...|[0.98865397199563...|       0.0|
|  0.0|[1.0,0.0,2.0,0.0,...|[0.52088456476688...|[18.8574368007347...|[0.94287184003673...|       0.0|
|  0.0|[1.0,0.0,2.0,0.0,...|[0.52088456476688...|[18.8574368007347...|[0.94287184003673...|       0.0|
|  0.0|[1.0,0.0,2.0,1.0,...|[0.52088456476688...|[19.6595901989730...|[0.98297950994865...|       0.0|
|  0.0|[1.0,0.0,2.0,1.0,...|[0.52088456476688...|[19.6595901989730...|[0.98297950994865...|       0.0|
|  0.0|[1.0,0.0,2.0,1.0,...|[0.52088456476688...|[19.6360052933126...|[0.98180026466563...|       0.0|
+-----+--------------------+--------------------+--------------------+--------------------+----------+
```

ml_11.py

Saving the model obtained from the pipeline:

```
pipe_t.save("model")
```



Metrics for Model Evaluation in **Apache Spark**

Metrics for Model Evaluation in **Apache Spark**

Key Metrics:

Accuracy ⇨ Proportion of correct predictions to total examples.

$$\text{Accuracy} = \frac{\text{Correct Predictions}}{\text{Total Examples}}$$

Precision ⇨ Proportion of true positives to total predicted positives for a class.

$$\text{Precision}_i = \frac{\text{True Positives}_i}{\text{True Positives}_i + \text{False Positives}_i}$$

Recall (Sensitivity): ⇨ Proportion of true positives to total actual positives for a class.

$$\text{Recall}_i = \frac{\text{True Positives}_i}{\text{True Positives}_i + \text{False Negatives}_i}$$

F1-Score: ⇨ Harmonic mean of Precision and Recall.

$$F1_i = 2 \cdot \frac{\text{Precision}_i \cdot \text{Recall}_i}{\text{Precision}_i + \text{Recall}_i}$$

Confusion Matrix: ⇨ A summary table showing true labels vs predicted labels for each class.

Example Confusion Matrix for Two Classes:

	Predicted: A	**Predicted: B**
Actual: A	True Positive (TP)	False Negative (FN)
Actual: B	False Positive (FP)	True Negative (TN)

Accuracy

Precision

Recall (Sensitivity):

F1-Score:

```python
from pyspark.ml import evaluation
from pyspark.sql import SparkSession # Import SparkSession

# Ensure SparkSession is active. This will get the existing session or create a new one.
spark = SparkSession.builder.appName("ml").getOrCreate()

# Accuracy (already calculated, but can be explicitly shown again)
evaluator_accuracy = evaluation.MulticlassClassificationEvaluator(metricName="accuracy")
accuracy = evaluator_accuracy.evaluate(pred_eval)
print(f"Accuracy: {accuracy:.4f}")

# Precision (weighted)
evaluator_precision = evaluation.MulticlassClassificationEvaluator(metricName="weightedPrecision")
precision = evaluator_precision.evaluate(pred_eval)
print(f"Weighted Precision: {precision:.4f}")

# Recall (weighted)
evaluator_recall = evaluation.MulticlassClassificationEvaluator(metricName="weightedRecall")
recall = evaluator_recall.evaluate(pred_eval)
print(f"Weighted Recall: {recall:.4f}")

# F1-Score (weighted)
evaluator_f1 = evaluation.MulticlassClassificationEvaluator(metricName="weightedFMeasure")
f1_score = evaluator_f1.evaluate(pred_eval)
print(f"Weighted F1-Score: {f1_score:.4f}")
```



```
Accuracy: 0.7595
Weighted Precision: 0.7886
Weighted Recall: 0.7595
Weighted F1-Score: 0.7614
```

Confusion Matrix: ⇨ A summary table showing true labels vs predicted labels for each class.

Example Confusion Matrix for Two Classes:

	Predicted: A	Predicted: B
Actual: A	True Positive (TP)	False Negative (FN)
Actual: B	False Positive (FP)	True Negative (TN)

Confusion matrix → is a fundamental tool for evaluating the performance of a classification model. It shows how often the model makes correct and incorrect predictions—broken down into specific categories.

What does a confusion matrix show?

Example:
Suppose a model classifies whether an email is spam.
After testing on 100 messages, we get:

	Actual Spam	Actual Not Spam
Predicted Spam	40	5
Predicted Not Spam	10	45

This means:
- **TP = 40** – correctly detected spam
- **TN = 45** – correctly recognized non-spam
- **FP = 5** – regular emails incorrectly marked as spam
- **FN = 10** – spam messages the model failed to detect

Why is a confusion matrix important?

Because accuracy alone can be misleading.

For example, if 95% of emails are non-spam, a model that always says “not spam” will have 95% accuracy… while completely failing to detect spam.
Only the confusion matrix reveals such error patterns.

```
from pyspark.sql.functions import col
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Select actual labels and predicted labels
predictions_and_labels = pred_eval.select(col("prediction").cast("integer"), col("label").cast("integer"))

# Calculate the confusion matrix using crosstab
confusion_matrix_df = predictions_and_labels.crosstab("label", "prediction")

# Convert to Pandas DataFrame for easier visualization
confusion_matrix_pd = confusion_matrix_df.toPandas()

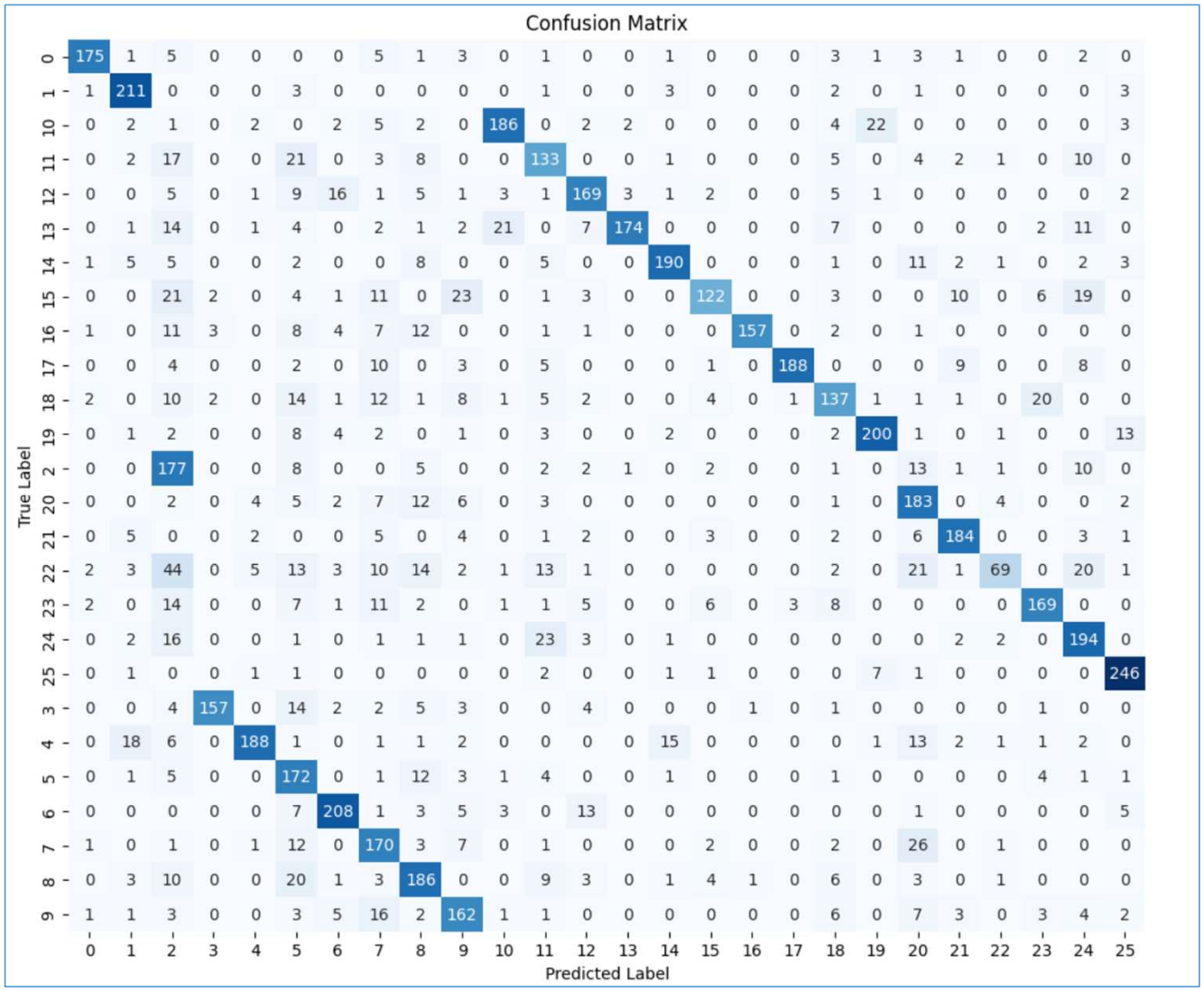
# Reorder columns to match the index (labels) for a proper confusion matrix view
# The column names from crosstab are strings, so we need to convert them to int to sort properly
# Exclude the 'label_prediction' column which contains the true labels
column_order = sorted([int(c) for c in confusion_matrix_pd.columns if c != 'label_prediction'])
column_order_str = [str(c) for c in column_order]

# Ensure 'label_prediction' column is first, then ordered prediction columns
confusion_matrix_pd = confusion_matrix_pd[['label_prediction'] + column_order_str]

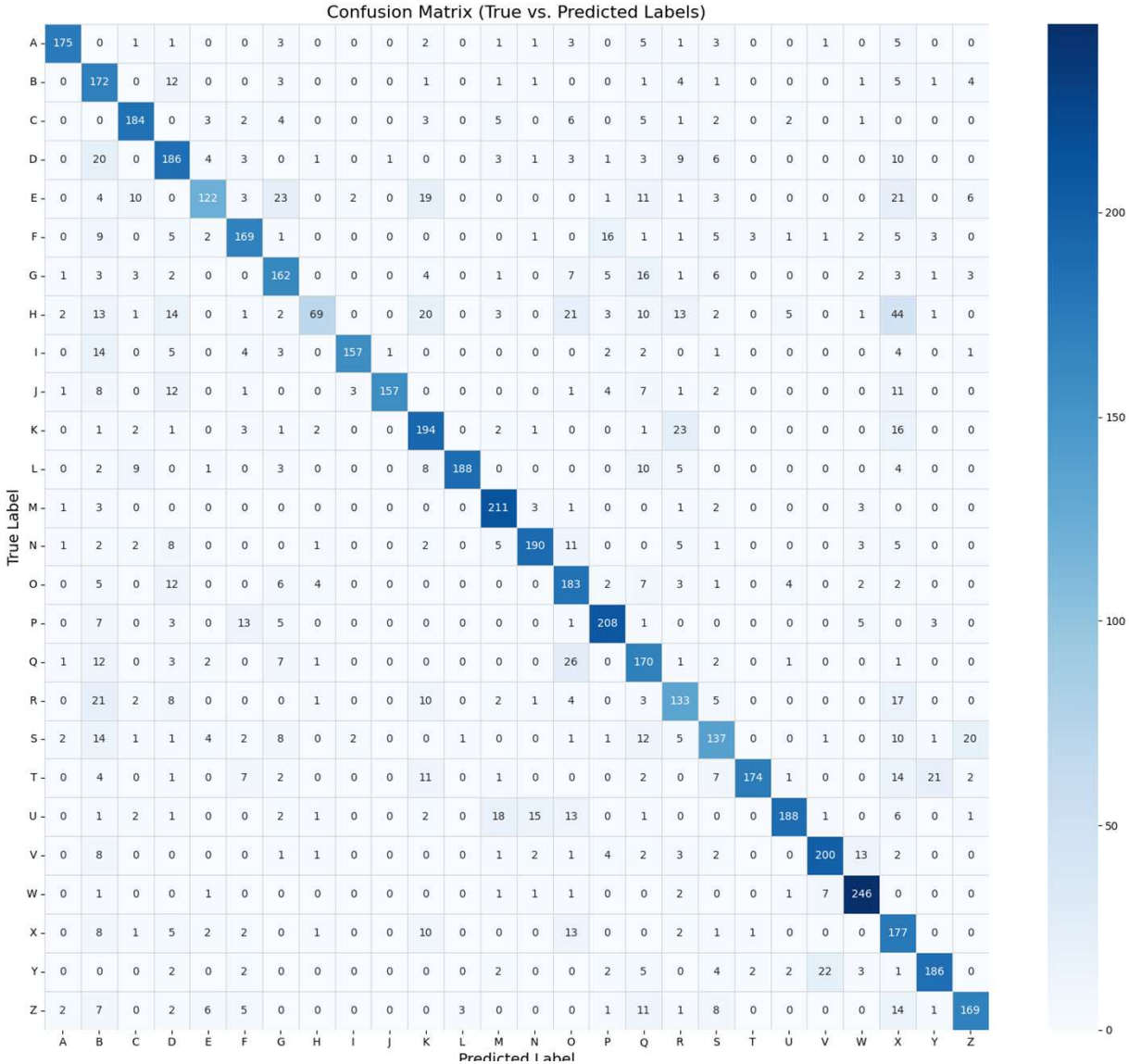
# Set 'label_prediction' column as index
confusion_matrix_pd = confusion_matrix_pd.set_index('label_prediction')

# Sort the index to match the column order for consistent display
confusion_matrix_pd = confusion_matrix_pd.sort_index()

print("Confusion Matrix:")
display(confusion_matrix_pd)
# Visualize the confusion matrix
plt.figure(figsize=(12, 10))
sns.heatmap(confusion_matrix_pd, annot=True, fmt="d", cmap="Blues", cbar=False)
plt.title("Confusion Matrix")
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.show()
```


Confusion Matrix
True Label
Predicted Label
0 1 10 11 12 13 14 15 16 17 18 19 2 20 21 22 23 24 25 3 4 5 6 7 8 9
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25

Retrieve Label-to-Letter Mapping




```
Numerical label for 'W': 25.0
Number of False Positives for 'W': 36
Number of False Negatives for 'W': 15
Number of True Positives for 'W': 246

False Positives for 'W' (top 5):
+-----+--------------------+--------------------+--------------------+--------------------+----------+
|label|                feat|            features|       rawPrediction|         probability|prediction|
+-----+--------------------+--------------------+--------------------+--------------------+----------+
|  5.0|[6.0,11.0,8.0,8.0...|[3.12530738860132...|[0.19046792424942...|[0.00952339621247...|      25.0|
| 21.0|[5.0,9.0,7.0,7.0,...|[2.60442282383443...|[0.39110646937464...|[0.01955532346873...|      25.0|
| 12.0|[3.0,6.0,6.0,4.0,...|[1.56265369430066...|[0.24436932365567...|[0.01221846618278...|      25.0|
| 12.0|[4.0,8.0,6.0,6.0,...|[2.08353825906754...|[0.05995553379821...|[0.00299777668991...|      25.0|
|  9.0|[5.0,9.0,7.0,8.0,...|[2.60442282383443...|[0.16144265324992...|[0.00807213266249...|      25.0|
+-----+--------------------+--------------------+--------------------+--------------------+----------+
only showing top 5 rows

False Negatives for 'W' (top 5):
+-----+--------------------+--------------------+--------------------+--------------------+----------+
|label|                feat|            features|       rawPrediction|         probability|prediction|
+-----+--------------------+--------------------+--------------------+--------------------+----------+
| 25.0|[1.0,0.0,2.0,0.0,...|[0.52088456476688...|[0.16800960893750...|[0.00840048044687...|      14.0|
| 25.0|[3.0,3.0,4.0,1.0,...|[1.56265369430066...|[0.01872659176029...|[9.36329588014981...|      19.0|
| 25.0|[3.0,4.0,4.0,3.0,...|[1.56265369430066...|[0.01872659176029...|[9.36329588014981...|      19.0|
| 25.0|[3.0,6.0,5.0,4.0,...|[1.56265369430066...|[0.15982661711541...|[0.00799133085577...|       4.0|
| 25.0|[4.0,5.0,4.0,3.0,...|[2.08353825906754...|[0.01872659176029...|[9.36329588014981...|      19.0|
+-----+--------------------+--------------------+--------------------+--------------------+----------+
only showing top 5 rows

True Positives for 'W' (top 5):
+-----+--------------------+--------------------+--------------------+--------------------+----------+
|label|                feat|            features|       rawPrediction|         probability|prediction|
+-----+--------------------+--------------------+--------------------+--------------------+----------+
| 25.0|[1.0,0.0,1.0,0.0,...|[0.52088456476688...|[0.16969169472388...|[0.00848458473619...|      25.0|
| 25.0|[1.0,0.0,2.0,0.0,...|[0.52088456476688...|[0.08874844456995...|[0.00443742222849...|      25.0|
| 25.0|[1.0,0.0,2.0,0.0,...|[0.52088456476688...|[0.08874844456995...|[0.00443742222849...|      25.0|
| 25.0|[1.0,0.0,2.0,1.0,...|[0.52088456476688...|[0.07002185280965...|[0.00350109264048...|      25.0|
| 25.0|[2.0,0.0,2.0,0.0,...|[1.04176912953377...|[0.17382289447781...|[0.00869114472389...|      25.0|
+-----+--------------------+--------------------+--------------------+--------------------+----------+
only showing top 5 rows
```

Conclusions:

High Accuracy for Many Letters:
Many letters show strong diagonal values, indicating that the model is generally good at recognizing them. For example, 'A' is correctly identified 175 times, 'C' 184 times, 'M' 211 times, 'T' 174 times, 'U' 188 times, and 'W' 246 times. These are among the most accurately classified letters.

'D' and 'B': True 'D' is sometimes predicted as 'B' (20 instances), and true 'B' is sometimes predicted as 'D' (12 instances). These letters might share similar visual characteristics or structural features after processing.
'E' and 'G': True 'E' is misclassified as 'G' 23 times, and true 'G' is misclassified as 'E' 23 times. This suggests a significant overlap in their feature representations.
'X' and 'H': True 'X' is predicted as 'H' 44 times, and true 'H' is predicted as 'X' 14 times. This is a notable one-sided confusion.
'Y' and 'V': True 'Y' is often misclassified as 'V' (22 times), which might be due to visual similarities, especially in certain handwriting styles or feature extractions. True 'V' is also confused with 'Y' (13 times).
'S' and 'Z': True 'S' is predicted as 'Z' 20 times, and true 'Z' is predicted as 'S' 8 times, showing another pattern of confusion.
'R' and 'K': True 'R' is often mistaken for 'K' (23 times), and true 'K' for 'R' (10 times).
Letters with Lower Performance: Some letters have a higher number of misclassifications compared to their correct predictions, indicating a lower recall or precision for these classes. For example, 'H' is correctly classified 69 times but misclassified as 'X' 44 times, 'R' 13 times, and 'D' 14 times, suggesting it's a challenging letter for the model.

Litery o niższej skuteczności: Niektóre litery mają większą liczbę błędnych klasyfikacji w porównaniu z ich prawidłowymi przewidywaniami, co wskazuje na niższą rozpoznawalność lub precyzję dla tych klas. Na przykład litera „H” została poprawnie sklasyfikowana 69 razy, ale błędnie sklasyfikowana jako „X” 44 razy, „R” 13 razy i „D” 14 razy, co sugeruje, że jest to trudna litera dla modelu.

Based on the random forest model, explain which input features are dominant. Provide a graph of the dominant features.

```
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd

# Get feature importances from the trained Random Forest model
# forest_t is the trained model from the earlier Random Forest classification cell
feature_importances = forest_t.featureImportances

# Get the original input column names that were used to create the 'features' vector
# These are stored in the 'inputCols' of the VectorAssembler 'assembler'
feature_names = assembler.getInputCols()

# Create a list of (feature_name, importance_score) tuples
feature_importance_list = []
for i, score in enumerate(feature_importances):
    feature_importance_list.append((feature_names[i], score))

# Sort the features by importance score in descending order
feature_importance_list.sort(key=lambda x: x[1], reverse=True)

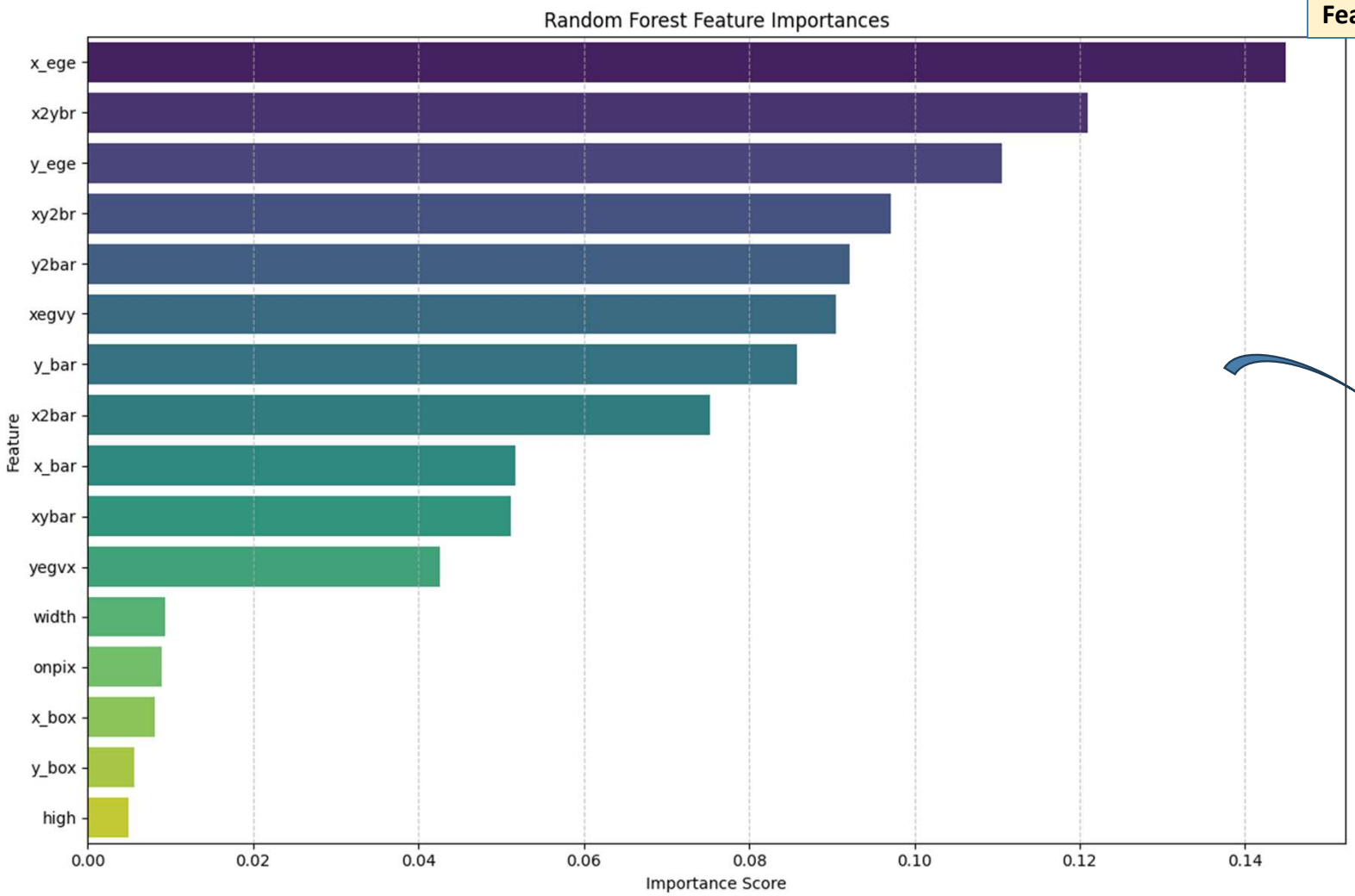
# Convert to a Pandas DataFrame for easier plotting
feature_importance_df = pd.DataFrame(feature_importance_list, columns=['Feature', 'Importance'])

print("Dominant Features based on Random Forest:")
display(feature_importance_df)

# Visualize the feature importances
plt.figure(figsize=(12, 8))
sns.barplot(x='Importance', y='Feature', data=feature_importance_df, hue='Feature', legend=False,
palette='viridis')
plt.title('Random Forest Feature Importances')
plt.xlabel('Importance Score')
plt.ylabel('Feature')
plt.grid(axis='x', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
```

Feature Importance from Random Forest Model:

index	Feature	Importance
0	x_ege	0.14489172734001957
1	x2ybr	0.12104447831754121
2	y_ege	0.11057053377584848
3	xy2br	0.09716291849462359
4	y2bar	0.09222324242126517
5	xegvy	0.0905742834228653
6	y_bar	0.08587033686728883
7	x2bar	0.07528190162394406
8	x_bar	0.05167393793288494
9	xybar	0.051229233543014195
10	yegvx	0.04254984785716752
11	width	0.009346490853400778
12	onpix	0.008905975641510213
13	x_box	0.00814850061328961
14	y_box	0.005641546555211108
15	high	0.004885044740



Feature Importance from Random Forest Model:

index	Feature	Importance
0	x_ege	0.14489172734001957
1	x2ybr	0.12104447831754121
2	y_ege	0.11057053377584848
3	xy2br	0.09716291849462359
4	y2bar	0.09222324242126517
5	xegvy	0.0905742834228653
6	y_bar	0.08587033686728883
7	x2bar	0.07528190162394406
8	x_bar	0.05167393793288494
9	xybar	0.051229233543014195
10	yegvx	0.04254984785716752
11	width	0.009346490853400778
12	onpix	0.008905975641510213
13	x_box	0.00814850061328961
14	y_box	0.005641546555211108
15	high	0.004885044740

Explain the meaning of the features `**x_ege**` and `**x2ybr**` in the context of letter recognition

Vide next

Explain the meaning of the features `**x_ege**` and `**x2ybr**` in the context of letter recognition

x_ege

pixel indicates a transition from background to foreground (or vice-versa), essentially counting how many times the horizontal scan lines cross the letter's stroke in that region. * **Why it's significant:** Horizontal edge information is extremely powerful for letter recognition. Consider: * **Letters with strong horizontal strokes:** 'T', 'E', 'F', 'H', 'A' (crossbar). These would likely have a higher `x_ege` value in certain regions. * **Letters with few or no horizontal strokes:** 'I', 'L', 'V', 'J'. These would tend to have lower `x_ege` values. The precise count and distribution of these horizontal edges help the model differentiate between characters that might otherwise look similar in terms of overall width or height. For instance, 'E' and 'F' are distinct primarily due to the number of horizontal segments.

x2ybr

is likely a normalized measure involving the squared x-coordinate and the y-coordinate of 'on' pixels. It could indicate the average squared x-position multiplied by the y-position for the 'on' pixels, or a similar cross-moment. * **Why it's significant:** These cross-moments provide rich information about the **shape, curvature, and density distribution** of the letter's pixels. They capture how pixels are spread out both horizontally and vertically simultaneously. For example: * A letter with more pixels in the top-right vs. bottom-left would have different x2ybr values. * Curved letters (like 'S', 'C', 'G') versus straight-lined letters (like 'I', 'L', 'T') will have distinct distributions of their squared x- and y-coordinates, which x2ybr helps to quantify. * It helps to capture subtleties in stroke orientation and how the mass of the letter is distributed across its area. Its high importance suggests that the model effectively leverages this detailed pixel distribution information to understand the intricate forms of different letters, helping to distinguish even visually complex or similar characters.

Based on the confusion matrix with letter labels → explain some strategies to improve your letter recognition model

Try a different classification algorithm

Tuning Hyperparameters in Apache Spark

Tuning Hyperparameters in Apache Spark

Hyperparameter tuning → in the context of machine learning models in Apache Spark is the process of optimizing the hyperparameter values of a model to improve its performance. Hyperparameters are model settings that are defined before the training process begins, such as the number of trees in random forests, the learning rate in gradient descent algorithms, or the number of iterations.

Apache Spark → offers built-in mechanisms for tuning hyperparameters in the MLlib module. There are primarily two main methods used:

1. **Grid Search (wyszukiwanie siatką)** → It involves defining a set of hyperparameter values to be tested. Spark iterates through all possible combinations of these values and evaluates each one, choosing the best one.

Example: If you have two hyperparameters (e.g. number of trees and maximum depth), you can specify a grid, e.g.

number of trees: [10, 50, 100]
max depth: [5, 10, 20]
Spark will test all combinations of these values, so: → 3×3=9 configurations.

2. **Random Search (losowe wyszukiwanie)** → Instead of testing all combinations, Spark randomly samples selected sets of hyperparameters, which saves time with larger search spaces.

Tuning Hyperparameters in Apache Spark

Hyperparameter tuning → is crucial for optimizing model performance.

For the Random Forest model, we can use PySpark's **`CrossValidator`** with a **`ParamGridBuilder`** to systematically search for the best hyperparameters. Here's a plan to do that:

→ **Define Hyperparameter Search Space**: Define the grid of hyperparameters to search over for the RandomForestClassifier. This includes parameters like `maxDepth`, `numTrees`, and `minInstancesPerNode`. Use `ParamGridBuilder` to create all combinations.

→ **Configure Cross-Validation**: Set up a `CrossValidator` with the RandomForestClassifier estimator, the defined hyperparameter grid, and a `MulticlassClassificationEvaluator` to assess model performance (e.g., using 'accuracy'). Specify the number of folds for cross-validation.

→ **Run Hyperparameter Tuning**: Execute the cross-validation process by fitting the `CrossValidator` to the training data (`df_train_`). This will train and evaluate multiple models across different hyperparameter combinations to find the best performing one.

→ **Retrieve Best Model and Evaluate**: Extract the best model found by the `CrossValidator`. Then, use this best model to make predictions on the evaluation dataset (`df_eval_`) and calculate its performance metrics (Accuracy, Precision, Recall, F1-Score) to compare with the initial model.

→ **Final Task**: Summarize the results of the hyperparameter tuning process, including the best hyperparameters found and the performance improvements (if any) compared to the default Random Forest model.

How does tuning work in Apache Spark?

How does tuning work in Apache Spark?

1. **Create a model** with hyperparameters for tuningThe model, such as Random Forest, must be configured to accept different hyperparameter values.

2. **Define the grid or hyperparameter space** → use a *ParamGridBuilder* object to define a set of values to test.

3. **Evaluate model performance** → use a metric such as accuracy, *F1-score*, or *RMSE*, depending on the problem. Spark uses an Evaluator object, such as *BinaryClassificationEvaluator* for binary classification.

4. **Use Cross-Validation or Train-Validation** Split Spark automatically splits the data into training and validation sets using a *CrossValidator* or *TrainValidationSplit* object.

PySpark code example:

How does tuning work in Apache Spark?

PySpark code example:

```
from pyspark.ml.tuning import ParamGridBuilder, CrossValidator
from pyspark.ml.evaluation import MulticlassClassificationEvaluator
from pyspark.ml.classification import RandomForestClassifier

# Model:
rf = RandomForestClassifier(labelCol="label", featuresCol="features")

# Siatka hiperparametrów
paramGrid = (ParamGridBuilder()
        .addGrid(rf.numTrees, [10, 50, 100])
        .addGrid(rf.maxDepth, [5, 10, 20])
        .build())

# Ewaluator:
evaluator = MulticlassClassificationEvaluator(metricName="accuracy")

# Cross-validation
cv = CrossValidator(estimator=rf,
            estimatorParamMaps=paramGrid,
            evaluator=evaluator,
            numFolds=5)  # 5-fold cross-validation

# Dopasowanie modelu:
cvModel = cv.fit(trainingData)

# Najlepsze hiperparametry
bestModel = cvModel.bestModel
print("Liczba drzew:", bestModel.getNumTrees)
print("Maksymalna głębokość:", bestModel.getMaxDepth())
```

Tuning Hyperparameters in Apache Spark